ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Приглашенный преподаватель	Академический руководитель образовательной программы «Программная инженерия», старший преподаватель департамента программной инженерии
_____ С. Б. Гурин	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 51 01-1-ЛУ

Исполнитель:
Студент группы БПИ236
_____ / Т. А. Петянов /
«___» _____ 2026 г.

Москва 2026

УТВЕРЖДЕН

RU.17701729.06.05-01 51 01-1-ЛУ

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

RU.17701729.06.05-01 51 01-1

Листов 30

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Москва 2026

АННОТАЦИЯ

Настоящий программный документ представляет собой сводную программу и методику испытаний программного комплекса «Sentinel — Интеллектуальный тайм-менеджер», включающего iOS-клиент, серверную часть и их сетевое взаимодействие.

В документе определены объект и цель испытаний, состав проверяемых требований к клиентской и серверной частям, контракт их взаимодействия, используемые технические и программные средства, порядок проведения испытаний и методы проверки функциональности, надежности и программной документации.

Особое внимание уделено сквозному пользовательскому сценарию: формированию календарных событий из свободного текстового запроса в чате, передаче вложений, получению структурированных предложений от серверной части, подтверждению действий пользователем и последующей синхронизации событий с календарём устройства.

Структура документа построена на основе уже используемого в проекте формата ПМИ и объединяет требования и методы испытаний, ранее описанные отдельно для iOS-клиента и backend-компонента.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.301-79 [7]: Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения к данному документу оформляются согласно ГОСТ 19.603-78 [8] и ГОСТ 19.604-78 [9].

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ . . . 5
1.1. Наименование программы . . . 5
1.2. Краткая характеристика области применения программы . . . 5
2. ЦЕЛЬ ИСПЫТАНИЙ . . . 6
3. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 7
3.1. Требования к функциональным характеристикам . . . 7
3.1.1. Требования к составу выполняемых функций . . . 7
3.1.2. Требования к организации входных данных . . . 9
3.1.3. Требования к организации выходных данных . . . 10
3.1.4. Требования к временным характеристикам . . . 11
3.1.5. Требования к интерфейсу . . . 11
3.1.6. Требования к хранению данных . . . 12
3.1.7. Интерфейсы API и взаимодействия компонентов . . . 12
3.2. Требования к надежности . . . 14
3.2.1. Требования к обеспечению надежного функционирования программы . . . 14
3.2.2. Время восстановления после отказа . . . 14
3.2.3. Отказы из-за некорректных действий оператора . . . 15
3.3. Условия эксплуатации . . . 15
3.3.1. Климатические условия эксплуатации . . . 15
3.3.2. Требования к видам обслуживания . . . 16
3.3.3. Требования к численности и квалификации персонала . . . 16
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ . . . 17
4.1. Состав программной документации . . . 17
4.2. Специальные требования к программной документации . . . 17
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ . . . 18
5.1. Состав технических и программных средств . . . 18
5.2. Порядок проведения испытаний . . . 18
6. МЕТОДЫ ИСПЫТАНИЙ . . . 20
6.1. Проверка требований к программной документации . . . 20
6.2. Unit-тестирование серверной части . . . 20
6.3. Интеграционное тестирование серверной части . . . 21

6.4. Автоматизированное тестирование iOS-клиента 22
6.5. Сквозное функциональное тестирование комплекса 22
6.6. Проверка требований к надежности 24
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 26
ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ 28
ПРИЛОЖЕНИЕ 3. РЕЗУЛЬТАТЫ АВТОМАТИЧЕСКОГО ТЕСТИРОВАНИЯ BACKEND-КОМПОНЕНТА 30

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Sentinel — Интеллектуальный тайм-менеджер».

Наименование программы на английском языке: «Sentinel — AI Scheduler».

Краткое наименование программы для пользователя: «Sentinel».

1.2. Краткая характеристика области применения программы

Программный комплекс Sentinel представляет собой клиент-серверную систему для интеллектуального управления личным расписанием пользователя. Система включает мобильное iOS-приложение, серверную часть с REST API и интеграцию с внешней языковой моделью, а также локальную синхронизацию с календарём устройства через EventKit.

iOS-клиент обеспечивает пользовательский интерфейс, работу с чатами, отображение предложенных событий, редактирование календарных сущностей, локальное хранение части пользовательского состояния, просмотр достижений, работу с профилем и правовыми документами.

Серверная часть обеспечивает аутентификацию, хранение пользователей, событий, чатов и достижений, интерпретацию пользовательских запросов с помощью LLM, загрузку изображений, ребалансировку расписания, формирование структурированных предложений действий и выдачу данных по контрактам REST API.

Сквозной сценарий работы системы состоит в следующем: пользователь формулирует запрос на естественном языке в iOS-приложении, при необходимости прикладывает изображения, серверная часть интерпретирует запрос, формирует предложения событий, пользователь подтверждает выбранные действия, после чего изменения сохраняются на сервере и синхронизируются с календарём устройства.

Программный комплекс может применяться в следующих сценариях:

1. Планирование встреч, занятий, дедлайнов и личных дел на основе свободного текстового описания.
2. Работа с изображениями как с входными данными для выделения задач, дат и временных ограничений.
3. Просмотр, подтверждение и отклонение предложенных системой изменений расписания.
4. Локальное редактирование событий и напоминаний с последующей синхронизацией состояния.
5. Автоматическая ребалансировка расписания на основе признака fixed и модели «Resource Battery».
6. Ведение истории диалогов, пользовательских настроек и прогресса в достижениях.

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия программного комплекса Sentinel требованиям технического задания, подтверждение корректности работы iOS-клиента и серверной части как по отдельности, так и в совместном режиме, а также подтверждение того, что программная документация, автоматизированные проверки и фактическая реализация образуют согласованный комплект, пригодный для приемки и дальнейшего сопровождения.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Требования к составу выполняемых функций

1. Функции iOS-клиента

1.1. Отображение экрана «Сегодня» для авторизованного и неавторизованного пользователя, включая карточки расписания, «Resource Battery», переходы к календарю, ребалансировке, профилю и достижениям.

1.2. Работа с диалогами: создание чата, загрузка списка чатов, выбор активного диалога, подгрузка более ранних сообщений, сохранение состояния активного чата, повторная отправка или удаление ошибочно неотправленного сообщения.

1.3. Работа с вложениями: показ панели выбора вложений, доступ к recent photos, переход к полному выбору изображений, использование камеры, системного файлового пикера, проверка типа, размера и количества вложений, запрос однократного согласия на серверную обработку вложений.

1.4. Работа с предложениями событий: отображение блока предложений в сообщении ассистента, разворачивание и сворачивание блока, выбор одного или нескольких предложений, индикация конфликтов, подтверждение выбранных действий и синхронизация состояния после применения.

1.5. Работа с календарём: отображение недельного и месячного представления, выбор дня, показ секций событий по дням, отображение day-level «Resource Battery», создание, открытие, редактирование и удаление событий и напоминаний.

1.6. Работа с профилем и настройками: отображение данных пользователя, редактирование Default Prompt, автоматическое сохранение Default Prompt, добавление Default Prompt к чатовым и rebalance-запросам, управление legal-flow, выход из учётной записи, удаление учётной записи, debug-only выбор окружения в тестовой сборке.

1.7. Работа с достижениями: отображение агрегированного прогресса, групп достижений и уровней, повторная загрузка при ошибке, возможность поделиться разблокированным достижением.

1.8. Работа со сценариями авторизации: вход, двухшаговая регистрация, подтверждение почты, повторная отправка письма подтверждения, восстановление пароля.

1.9. Работа со сценарием ребалансировки: выбор дней, загрузка и отображение preview, refresh preview, apply изменений и закрытие сценария.

2. Функции серверной части

2.1. Регистрация пользователя, вход, выдача JWT, подтверждение email, восстановление пароля,

обновление профиля и удаление учётной записи.

2.2. Хранение и выдача чатов и сообщений с cursor-based пагинацией.

2.3. Приём пользовательских сообщений, оркестрация взаимодействия с внешней языковой моделью и возврат ответа ассистента.

2.4. Загрузка изображений в S3-совместимое хранилище и использование их URL в multimodal-сценарии LLM.

2.5. Поддержка паттерна propose-before-apply: сервер формирует предложения действий над событиями со статусом pending, а фактическое применение выполняет только после явного подтверждения пользователя.

2.6. CRUD-операции над событиями и напоминаниями, поиск событий по диапазону дат, идемпотентное создание событий по заголовку X-Idempotency-Key, пакетная офлайн-синхронизация до 500 операций.

2.7. Dry-run и apply ребалансировки расписания с учётом is_fixed и «Resource Battery», валидацией ответа LLM и одним повтором при невалидном ответе.

2.8. Выдача достижений на основе счётчиков активности пользователя.

2.9. Предоставление OpenAPI 3.x документации, health-check и базовой защиты от злоупотреблений с помощью rate limiting.

3. Функции взаимодействия компонентов

3.1. Обмен данными между iOS-клиентом и серверной частью должен выполняться по REST API с префиксом `/api/v1` в формате JSON по HTTPS, а для загрузки изображений — в формате multipart/form-data.

3.2. Аутентификация между клиентом и сервером должна выполняться через JWT в заголовке `Authorization: Bearer <token>`.

3.3. Клиент должен передавать на сервер сообщения пользователя, данные профиля, параметры календарных сущностей, запросы на apply предложений и запросы на ребалансировку.

3.4. Сервер должен возвращать клиенту структурированный результат в виде JSON, пригодный для визуализации предложений событий, обновления истории чата и синхронизации состояния календаря.

3.5. После применения предложений или ребалансировки клиент должен повторно синхронизировать календарные данные и отдельно поддерживать экспорт в EventKit без дублирования событий.

3.6. Серверное состояние событий и локальная синхронизация EventKit рассматриваются как

отдельные контуры; наличие локального события не должно подменять серверный источник истины.

3.7. Часовые пояса пользователя должны передаваться и интерпретироваться согласованно: клиент отправляет даты и время в ISO-8601, сервер хранит временные значения в UTC и учитывает IANA-тайм-зону пользователя при логике LLM и профиля.

3.1.2. Требования к организации входных данных

Входные данные программного комплекса представляют собой:

1. Пользовательские данные iOS-клиента

1.1. Текстовые сообщения длиной до 2000 символов в кодировке UTF-8.

1.2. Изображения форматов PNG, JPG, JPEG и HEIC размером до 10 МБ на файл и в количестве до 10 файлов на сообщение.

1.3. Данные аутентификации: email, пароль, verification token, reset token.

1.4. Данные календарных сущностей: заголовок, описание, местоположение, дата и время начала, дата и время окончания, признаки all-day, fixed, reminder или event.

1.5. Данные профиля и контекста: Default Prompt, выбранные дни ребалансировки, временная зона устройства, выбранное окружение тестовой сборки.

2. Входные данные серверной части

2.1. JSON по HTTPS для всех основных бизнес-операций.

2.2. Multipart/form-data для загрузки изображений.

2.3. Заголовок Authorization с Bearer JWT для защищённых эндпоинтов.

2.4. Необязательный заголовок X-Idempotency-Key длиной до 255 символов при создании событий.

2.5. Даты и время в формате ISO-8601, интерпретируемые с учётом пользовательской тайм-зоны IANA.

2.6. Списки upserts и deletes для офлайн-синхронизации не более 500 элементов в каждом списке.

3. Требования к валидации входных данных

3.1. Проверка обязательности полей в auth-сценариях и сценариях удаления аккаунта.

3.2. Проверка типа, размера и количества вложений до начала отправки или загрузки.

3.3. Проверка обязательности непустого заголовка события.

3.4. Проверка условия `end_at > start_at` для событий с завершением.

3.5. Проверка длины `ai_instructions` и иных полей профиля на стороне сервера.

3.6. Проверка прав доступа пользователя к событиям, чатам, сообщениям и достижениям.

3.7. Запрос однократного согласия пользователя на серверную обработку вложений перед первой отправкой изображений или файлов.

3.1.3. Требования к организации выходных данных

1. Выходные данные iOS-клиента

1.1. Экранные представления, визуальные состояния загрузки, ошибки и действия повторной попытки.

1.2. Блоки предложенных событий, состояния pending, accepted и rejected, индикаторы конфликтов и данные для календарного отображения.

1.3. Экранные данные профиля, legal-flow, достижений, auth-flow и rebalance-flow.

2. Выходные данные серверной части

2.1. JSON-ответы по контрактам REST API с корректными HTTP-статусами 2xx, 4xx и 5xx.

2.2. Ответы поиска событий в формате `{items: [...], total: int}`.

2.3. Ответы по чатам и сообщениям в формате `{items: [...], has_more: bool}`.

2.4. Структурированный контент сообщений `content_structured`, включая `event_actions` и `image_message`.

2.5. Ответы ребалансировки в формате `{proposed: [...], summary: строка, changed_count: int, unchanged_count: int}`.

2.6. Ответы достижений в формате `{groups: [...]}`.

2.7. Единый формат ошибок `{code, message, details}`.

3. Постоянно сохраняемые данные

3.1. На клиенте: запись синхронизации календаря в SwiftData, пользовательская сессия в Keychain, AppSettings и service-флаги в UserDefaults.

3.2. На сервере: пользователи, события, чаты, сообщения, достижения, счётчики, токены и ключи идемпотентности в реляционной базе данных; изображения в объектном хранилище.

3.3. Во внешней системе: события календаря устройства, синхронизированные через EventKit.

4. Требования к целостности выходных данных

4.1. Исключение дублирования событий при экспорте в календарь устройства.

4.2. Сохранение связи между серверными идентификаторами событий и локальными идентификаторами EventKit.

4.3. Сохранение признака временного конфликта для предложенных и существующих событий.

4.4. Согласованность статусов предложений событий между серверной частью и отображением клиента после apply-операций.

4.5. Изоляция пользовательских данных: ни один пользователь не должен получать чужие события, чаты, достижения или токены.

3.1.4. Требования к временным характеристикам

1. Реакции интерфейса iOS-клиента, не требующие сетевого запроса, должны выполняться без визуально заметной задержки для пользователя.
2. Время ответа API для простых операций чтения списков и деталей должно составлять не более 500 мс при номинальной нагрузке.
3. Поиск событий на серверной стороне по временному диапазону должен иметь медианное время отклика не более 200-300 мс.
4. Целевое время обработки чатового запроса с участием LLM составляет 3-8 секунд; предельное ожидаемое время ответа не должно превышать 15 секунд.
5. Создание, обновление или удаление события через интерфейс клиента не должно приводить к блокировке UI и должно укладываться в нормальное пользовательское ожидание.
6. Dry-run ребалансировки должен сопровождаться явной индикацией ожидания, если выполнение занимает больше обычного пользовательского ожидания сетевой операции.
7. Автосохранение Default Prompt должно происходить при завершении редактирования без отдельной кнопки сохранения.

3.1.5. Требования к интерфейсу

1. Требования к интерфейсу iOS-клиента
    1.1. Соответствие Apple Human Interface Guidelines.
    1.2. Поддержка светлой и тёмной темы оформления.
    1.3. Поддержка Dynamic Type, VoiceOver и корректных accessibility-меток.
    1.4. Минимальный размер интерактивных элементов — 44×44 pt.
    1.5. Контраст элементов интерфейса не ниже требований WCAG 2.1 AA.
    1.6. Основной сценарий «чат → предложения событий → подтверждение → календарь» должен быть доступен без перехода в служебные разделы.
    1.7. Legal-flow должен открывать документы поверх приложения во встроенном Safari controller.
    1.8. Элемент Environment должен присутствовать только в тестовых сборках.
2. Требования к сетевому интерфейсу
    2.1. Версионирование REST API должно использовать префикс `/api/v1`.
    2.2. Формат обмена — JSON по HTTPS, для загрузки изображений — multipart/form-data.
    2.3. Идентификаторы доменных объектов — UUID v4.

2.4. Формат ошибок — единый для всех API-операций.

2.5. Все временные значения должны передаваться в ISO-8601.

3.1.6. Требования к хранению данных

Система должна обеспечивать надёжное хранение следующих сущностей и состояний:

1. Клиентские хранилища

1.1. SwiftData — записи синхронизации календаря и сопоставления локальных и серверных событий.

1.2. Keychain — пользовательская сессия и иные чувствительные данные аутентификации.

1.3. UserDefaults — AppSettings, Default Prompt, сервисные флаги и сведения о недавно активном чате.

2. Серверные сущности

2.1. Пользователи (`users`): UUID, email, хеш пароля, признак верификации, тайм-зона, локаль, preferences, ai_instructions, временные метки.

2.2. События и напоминания (`events`): UUID, связь с пользователем, заголовок, описание, start_at, end_at, all_day, type, location, is_fixed, source, временные метки.

2.3. Чаты (`chats`) и сообщения (`chat_messages`): UUID, связи с пользователем и чатом, роль, текстовое и структурированное содержимое, идентификатор модели ИИ, временные метки.

2.4. Достижения (`achievements`, `user_achievements`) и счётчики (`user_counters`).

2.5. Ключи идемпотентности (`idempotency_keys`) и auth-токены (`auth_tokens`).

3. Требования к организации хранения

3.1. Первичные ключи всех серверных сущностей — UUID v4.

3.2. Временные значения на сервере хранятся в UTC с использованием TIMESTAMPTZ.

3.3. Структурированные поля и preferences могут храниться в JSONB.

3.4. При удалении пользователя должно выполняться каскадное удаление всех его данных.

3.5. Должны поддерживаться ограничения уникальности для email, уровней достижений, выданных достижений и ключей идемпотентности.

3.1.7. Интерфейсы API и взаимодействия компонентов

1. Аутентификация и профиль

1.1. `POST /api/v1/auth/register` — регистрация пользователя.

1.2. `POST /api/v1/auth/login` — вход и выдача JWT.

1.3. `POST /api/v1/auth/verify-email` и `POST /api/v1/auth/resend-verification` — подтверждение email и повторная отправка письма.

1.4. `POST /api/v1/auth/forgot-password` и `POST /api/v1/auth/reset-password` — восстановление пароля.

1.5. `GET /api/v1/auth/me`, `PATCH /api/v1/auth/me`, `DELETE /api/v1/auth/me` — получение, изменение и удаление профиля.

2. Чаты, сообщения и изображения

2.1. `GET /api/v1/chats`, `POST /api/v1/chats`, `DELETE /api/v1/chats/{id}` — управление чатами.

2.2. `GET /api/v1/chats/{id}/messages`, `POST /api/v1/chats/{id}/messages` — чтение истории и отправка сообщения.

2.3. `POST /api/v1/chats/{id}/messages/{msg_id}/apply` — подтверждение или отклонение предложений действий.

2.4. `POST /api/v1/chats/{id}/upload` — загрузка изображения для multimodal-сценария.

3. События и напоминания

3.1. `GET /api/v1/events`, `POST /api/v1/events`, `GET /api/v1/events/{id}`, `PATCH /api/v1/events/{id}`, `DELETE /api/v1/events/{id}` — CRUD и поиск событий.

3.2. `POST /api/v1/events/sync` — пакетная офлайн-синхронизация.

3.3. `POST /api/v1/events/rebalance` и `POST /api/v1/events/rebalance/apply` — preview и применение ребалансировки.

4. Достижения и системные операции

4.1. `GET /api/v1/achievements` — получение групп достижений и прогресса.

4.2. `GET /api/v1/health` — проверка работоспособности сервиса.

5. Правила взаимодействия компонентов

5.1. Клиент не должен напрямую изменять локальный календарь до завершения серверного подтверждения и обновления серверного состояния.

5.2. Сервер не должен выполнять LLM-предложения без явного пользовательского apply-запроса.

5.3. После apply клиент обязан обновить данные чата и календаря и сохранить сопоставление локальных и серверных идентификаторов.

5.4. Повторные запросы создания событий с одним и тем же `X-Idempotency-Key` не должны приводить к дублированию серверных событий.

5.5. Экспорт в EventKit должен обеспечивать отсутствие локальных дублей даже при повторных синхронизациях или перезапусках клиента.

3.2. Требования к надежности

3.2.1. Требования к обеспечению надежного функционирования программы

1. Требования к надежности iOS-клиента

1.1. Валидация входных данных перед запуском критических операций.

1.2. Контролируемая обработка сетевых ошибок, ошибок API и локальных отказов без нарушения целостности пользовательского состояния.

1.3. Идемпотентность операций локальной синхронизации календаря для предотвращения дублирования событий.

1.4. Хранение токенов и иных чувствительных данных в Keychain.

1.5. Сохранение пользовательских настроек вне volatile-состояния UI.

1.6. Сохранение видимости конфликтов расписания и ошибок отправки сообщений.

2. Требования к надежности серверной части

2.1. Валидация входных данных через схемы сервиса и контроль прав доступа.

2.2. Транзакционность всех мутирующих операций с событиями, сообщениями, профилем и достижениями.

2.3. Поддержка идемпотентности при создании событий через `X-Idempotency-Key`.

2.4. Один повтор запроса к LLM при невалидном ответе в сценарии ребалансировки.

2.5. Graceful degradation: при недоступности LLM или хранилища изображений сервис должен возвращать контролируемую ошибку.

2.6. Rate limiting для чувствительных эндпоинтов.

2.7. Структурированное логирование HTTP-запросов и ключевых бизнес-операций.

3. Требования к надежности взаимодействия компонентов

3.1. Пользовательские действия не должны приводить к рассогласованию серверного состояния и локального календаря без возможности повторной синхронизации.

3.2. Источник истины для серверных событий — backend; локальная синхронизация EventKit должна только отражать согласованное серверное состояние.

3.3. Ошибки авторизации, валидации и доступа должны корректно передаваться клиенту и отображаться пользователю без аварийного завершения приложения.

3.2.2. Время восстановления после отказа

1. На клиенте после перезапуска приложения должны автоматически восстанавливаться пользовательская сессия, Default Prompt, выбранное окружение тестовой сборки и идентификатор недавно активного чата в пределах окна актуальности.

2. После восстановления сетевой доступности клиент должен иметь возможность повторно загрузить профиль, чаты, сообщения, события и достижения.

3. При отказе отдельного серверного компонента восстановление штатного режима работы должно обеспечиваться перезапуском контейнеров командой `docker compose restart`; целевое время восстановления — не более 5 минут.

4. При полном отказе сервера восстановление должно выполняться из резервной копии базы данных; целевое время восстановления — не более 30 минут при наличии актуальной резервной копии.

3.2.3. Отказы из-за некорректных действий оператора

1. Некорректные действия пользователя в auth-сценариях

1.1. Отсутствие обязательных полей должно приводить к валидационной ошибке без повреждения состояния.

1.2. Неверный пароль, несуществующий email, отсутствующий verification token или reset token должны обрабатываться контролируемо.

2. Некорректные действия пользователя при работе с сообщениями и вложениями

2.1. Превышение лимита размера или количества вложений должно приводить к отказу в отправке с понятным сообщением пользователю.

2.2. Ошибочно неотправленное сообщение должно оставаться в управляемом состоянии с действиями Retry или Remove.

3. Некорректные сетевые или серверные обращения

3.1. Невалидный JSON или нарушение схемы входных данных должны приводить к ошибке 422.

3.2. Отсутствующий или истёкший JWT должен приводить к ошибке 401.

3.3. Обращение к чужим ресурсам должно приводить к ошибке 403.

3.4. Обращение к несуществующим ресурсам должно приводить к ошибке 404.

3.5. Превышение rate limit должно приводить к ошибке 429.

4. Общий результат

4.1. Никакое некорректное действие пользователя или оператора не должно приводить к утрате данных других пользователей, нарушению целостности базы данных или неуправляемому аварийному завершению клиентского приложения.

3.3. Условия эксплуатации

3.3.1. Климатические условия эксплуатации

Для iOS-клиента применимы штатные условия эксплуатации мобильных устройств Apple. Для серверной части применимы стандартные условия эксплуатации серверного программного

обеспечения; климатические требования определяются условиями размещения серверной инфраструктуры и провайдера облака.

3.3.2. Требования к видам обслуживания

1. Регулярное обновление зависимостей iOS-проекта, серверных пакетов и системных компонентов окружения.
2. Мониторинг логов, основных метрик производительности и состояния health-check.
3. Резервное копирование базы данных и контроль возможности восстановления.
4. Контроль актуальности правовых документов, API-контрактов и конфигурации окружений.

3.3.3. Требования к численности и квалификации персонала

1. iOS-разработчик, владеющий Swift, SwiftUI, TCA, EventKit и средствами тестирования Xcode.
2. Backend-разработчик, владеющий разработкой веб-сервисов, реляционными СУБД, Docker и автоматизированным тестированием на Python.
3. При необходимости — DevOps-специалист для CI/CD, хранилища образов, контейнеризации и развёртывания.
4. Конечный пользователь специальных навыков администрирования не требует.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

4.1. Состав программной документации

1. «Sentinel — Интеллектуальный тайм-менеджер». Техническое задание (ГОСТ 19.201-78).
2. «Sentinel — Интеллектуальный тайм-менеджер». Программа и методика испытаний (ГОСТ 19.301-79).
3. «Sentinel — Интеллектуальный тайм-менеджер». Пояснительная записка (ГОСТ 19.404-79).
4. «Sentinel — Интеллектуальный тайм-менеджер». Руководство оператора (ГОСТ 19.505-79).
5. «Sentinel — Интеллектуальный тайм-менеджер». Текст программы (ГОСТ 19.401-78).

4.2. Специальные требования к программной документации

Документы к программному комплексу должны быть выполнены в соответствии с ГОСТ 19.106-78 и ГОСТами к каждому виду документа. Тексты документов должны быть согласованы между собой по наименованию программы, составу функций, архитектуре клиент-серверного взаимодействия, API-контрактам, платформенным ограничениям и описанию сквозных пользовательских сценариев.

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Состав технических и программных средств

Для проведения испытаний использовались следующие технические и программные средства:

1. Технические средства

1.1. Компьютер Apple Mac с установленной средой разработки Xcode.

1.2. Симулятор iPhone 17 Pro с версией iOS 26.2.

1.3. Серверное окружение с CPU не менее 2 vCPU, RAM не менее 4 ГБ, дисковым пространством не менее 20 ГБ и доступом в Интернет для LLM API.

1.4. Файловая система разработки с локальными копиями iOS-репозитория, backend-проекта и комплекта документации.

2. Программные средства

2.1. Xcode 26.2.

2.2. Схема проекта «Sentinel».

2.3. Swift Testing, TCA TestStore и xccov для клиентских автоматизированных проверок.

2.4. Docker 24.0+, Docker Compose 2.20+, Python 3.11+, pytest 8.0+, pytest-asyncio и httpx для backend-проверок.

2.5. PostgreSQL и объектное хранилище, используемые серверной частью.

2.6. Swagger UI / OpenAPI 3.x для ручных функциональных проверок API.

3. Испытуемые артефакты

3.1. Исходный код iOS-приложения Sentinel, включая основной таргет и локальный Swift package SentinelModules.

3.2. Исходный код серверной части Sentinel, включая API, сервисы, репозитории и миграции.

3.3. Комплект программной документации.

3.4. Автоматизированные тестовые наборы `SentinelTests`, `tests/unit/` и `tests/integration/`.

5.2. Порядок проведения испытаний

Испытания проводились в следующем порядке:

1. Анализ комплекта проектной документации и проверка согласованности терминологии клиента, сервера и интеграционного контура.
2. Подготовка серверного окружения, запуск контейнеров и проверка работоспособности health-check.

3. Запуск unit-тестирования серверной части.
4. Запуск интеграционного тестирования серверной части с подключением к базе данных.
5. Запуск автоматизированного тестового набора iOS-клиента на симуляторе iPhone 17 Pro с iOS 26.2 через Xcode 26.2.
6. Сопоставление результатов автоматизированных проверок с основными пользовательскими сценариями и API-контрактами.
7. Проведение сквозных функциональных проверок взаимодействия iOS-клиента и серверной части.
8. Проверка требований к надежности, восстановлению после отказов и обработке некорректных действий.
9. Фиксация результатов испытаний, выявленных ограничений и областей дальнейшего наращивания тестовой базы.

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к программной документации

Проверка требований к программной документации выполнялась путём анализа состава документов, согласованности их структуры и соответствия их содержания фактическому программному комплексу Sentinel.

1. Проверка состава документации

1.1. Проверено наличие технического задания.

1.2. Проверено наличие пояснительной записки.

1.3. Проверено наличие руководства оператора.

1.4. Проверено наличие текста программы.

1.5. Проверено наличие программы и методики испытаний.

2. Проверка согласованности наименований и терминологии

2.1. Проверено единое использование наименований «Sentinel — Интеллектуальный тайм-менеджер» и «Sentinel — AI Scheduler».

2.2. Проверено согласованное использование терминов «Profile», «Default Prompt», «Resource Battery», «Rebalance», «Environment», «JWT», «OpenAPI», «EventKit».

2.3. Проверено согласованное описание сценария «чат → предложения событий → подтверждение → календарь».

3. Проверка содержательной согласованности

3.1. Проверено соответствие клиентских экранов и состояний описанию в документации.

3.2. Проверено соответствие серверных эндпоинтов и форматов данных их описанию в документации.

3.3. Проверено, что взаимодействие клиента и сервера описано как единый сквозной сценарий, а не как два независимых продукта.

4. Результаты проверки

4.1. Комплект основных документов по проекту присутствует.

4.2. Наименования программы, интерфейсы и основные термины приведены к согласованному виду.

4.3. Настоящий документ синхронизирован с клиентским и серверным контурами реализации.

6.2. Unit-тестирование серверной части

Unit-тесты проверяют корректность изолированных серверных компонентов без взаимодействия с реальной базой данных или внешними сервисами. Для запуска используется `venv/bin/pytest tests/unit/ -v`.

1. Проверка безопасности и аутентификации: `test_security.py`, `test_auth_token_repository.py`.
2. Проверка Pydantic-схем и валидации событий: `test_schemas.py`.
3. Проверка LLM-оркестрации, tool-use и propose-before-apply: `test_llm_service.py`, `test_llm_vision.py`.
4. Проверка ребалансировки и валидации результатов LLM: `test_rebalance_service.py`.
5. Проверка репозиториев чатов, событий, пользователей, достижений, счётчиков и идемпотентности: `test_chat_repository.py`, `test_event_repository.py`, `test_user_repository.py`, `test_achievement_repository.py`, `test_counter_repository.py`, `test_idempotency_repository.py`.
6. Проверка инфраструктурных сервисов: `test_storage_service.py`, `test_email_service.py`.
7. Результат: в серверной части успешно выполнено 385 автоматических тестов; суммарное покрытие кода составило 87%.

6.3. Интеграционное тестирование серверной части

Интеграционные тесты проверяют работу API от HTTP-запроса до базы данных через реальное подключение к PostgreSQL. Для запуска используется `docker compose up -d`, затем `venv/bin/pytest tests/integration/ -v`.

1. `test_health.py` — проверка health-check.
2. `test_auth.py` и `test_profile.py` — регистрация, вход, профиль, удаление аккаунта, работа JWT.
3. `test_events.py` и `test_sync.py` — CRUD событий, поиск по диапазону, пакетная синхронизация, лимиты batch-операций.
4. `test_chats.py` и `test_images.py` — чаты, история сообщений, загрузка изображений, apply предложений, multimodal-поток.
5. `test_rebalance.py` — dry-run и apply ребалансировки, проверка is_fixed и валидации диапазонов.
6. `test_achievements.py` и `test_idempotency.py` — достижения и поведение при повторных create-запросах.
7. Результат: интеграционные проверки подтверждают корректность API-контрактов, изоляцию пользовательских данных, обработку ошибок 401/403/404/422 и корректную работу основных серверных бизнес-сценариев.

6.4. Автоматизированное тестирование iOS-клиента

Автоматизированные испытания iOS-клиента проводились штатными средствами Xcode по схеме «Sentinel» на симуляторе iPhone 17 Pro с iOS 26.2. Для написания проверок используется Swift Testing; для тестирования редьюсеров The Composable Architecture используется TestStore.

1. Тесты сетевых контрактов и транспортного слоя: `APIRequestAndErrorTests.swift`, `DTOCodingTests.swift`, `APIModelConverterTests.swift`.
2. Тесты сессии и настроек приложения: `SessionModelTests.swift`, `AppConfigurationAndSettingsTests.swift`, `DefaultPromptEnvelopeTests.swift`.
3. Тесты доменных моделей и производных состояний: `HomeStateDerivedTests.swift`, `CalendarStateDerivedTests.swift`, `AchievementAndAuthStateTests.swift`, `ChatDomainModelTests.swift`, `ChatThreadSupportTests.swift`.
4. Тесты auth-flow: `AuthReducerFlowTests.swift`.
5. Тесты чатового сценария и вложений: `ReducerFlowTests.swift`, `ChatThreadFeatureFlowTests.swift`, `ChatAttachmentPickerFeatureTests.swift`.
6. Тесты календаря и ребалансировки: `CalendarReducerFlowTests.swift`, `RebalanceFeatureFlowTests.swift`.
7. Тесты навигации и визуального слоя: `HomeProfileNavigationFeatureTests.swift`, `ViewSmokeTests.swift`.
8. Результат: успешно выполнено 82 теста, сгруппированных в 21 suite; все тесты завершились со статусом `TEST SUCCEEDED`; покрытие клиентского приложения по xccov составило 58,28% (8482 покрытых строк из 14555 исполняемых строк).
9. Ограничение: отдельный набор XCUITest-сценариев на момент испытаний отсутствует; полноценные end-to-end проверки камеры, системных разрешений и реального календаря устройства должны наращиваться в следующих итерациях.

6.5. Сквозное функциональное тестирование комплекса

Сквозные функциональные проверки выполнялись путём сопоставления клиентских flow-тестов, серверных интеграционных тестов, API-контрактов и логики фактического взаимодействия между компонентами.

1. Сценарий аутентификации и профиля

1.1. Проверялись регистрация, вход, получение профиля, обновление тайм-зоны и ai-instructions, выход и удаление аккаунта.

1.2. Ожидаемый результат: клиент корректно хранит JWT, сервер возвращает профиль текущего пользователя, после удаления аккаунта прежний JWT становится недействительным.

2. Сценарий отправки сообщения и вложений

2.1. Проверялись отправка текстового сообщения, автоматическое создание чата при необходимости, загрузка изображения, использование consent и появление multimodal-контента в истории чата.

2.2. Ожидаемый результат: клиент отображает сообщение в ленте, сервер сохраняет сообщение и возвращает ответ ассистента; изображение загружается в объектное хранилище и участвует в обработке запроса.

3. Сценарий предложений событий

3.1. Проверялись получение блока `event_actions`, выбор одного или нескольких предложений, отклонение части предложений и apply-операция.

3.2. Ожидаемый результат: до apply действия находятся в состоянии pending; после apply сервер сохраняет принятые изменения, а клиент обновляет сообщение ассистента и локальные календарные данные.

4. Сценарий синхронизации календаря

4.1. Проверялись создание и редактирование события на сервере, повторная загрузка его клиентом и локальный экспорт в EventKit без дублирования.

4.2. Ожидаемый результат: серверное событие отображается на клиенте, а повторные синхронизации не создают дублей в локальном календаре.

5. Сценарий ребалансировки

5.1. Проверялись загрузка доступных дней, передача Default Prompt и day-level данных, получение preview и apply изменений.

5.2. Ожидаемый результат: dry-run не изменяет серверные данные, apply изменяет только подтверждённые события, fixed-события не нарушаются.

6. Сценарий достижений

6.1. Проверялись накопление счётчиков на сервере, получение групп достижений клиентом и корректное отображение прогресса.

6.2. Ожидаемый результат: клиент отображает только данные текущего пользователя, повторная загрузка не создаёт ложных повторных выдач достижений.

7. Общий результат

7.1. Сквозной сценарий «чат → предложения событий → подтверждение → календарь»

подтверждён совокупностью серверных и клиентских автоматизированных проверок и соответствует основной продуктовой логике Sentinel.

7.2. Сквозной сценарий «профиль → Default Prompt → чат / ребалансировка» подтверждён: значение Default Prompt сохраняется на клиенте и учитывается при формировании сетевых запросов.

6.6. Проверка требований к надежности

Проверка надежности выполнялась на основе автоматизированных тестов серверных и клиентских модулей, а также анализа ветвей валидации, error-handling и восстановления состояния.

1. Проверка устойчивости к некорректному вводу

1.1. Проверялась обязательность email, пароля, verification token, reset token и заголовка события.

1.2. Проверялась реакция на слишком большой файл, превышение лимита вложений и невалидный JSON.

1.3. Результат: перечисленные ветви обрабатываются контролируемо и не приводят к аварийному завершению.

2. Проверка поведения при ошибках операций

2.1. Проверялась реакция на неуспешную отправку сообщения, ошибку сохранения события, ошибку удаления события и невалидный ответ LLM в сценарии ребалансировки.

2.2. Результат: ошибки переводят модули в предсказуемое состояние, сохраняют необходимые пользовательские данные и формируют читаемое сообщение об ошибке.

3. Проверка восстановления состояния

3.1. Проверялась попытка восстановления клиентской сессии при наличии и отсутствии сохранённых данных.

3.2. Проверялись повторная загрузка AppSettings, недавно активного чата и повторное получение серверных данных после перезапуска.

3.3. Проверялась возможность восстановления серверного приложения после перезапуска контейнера по ответу `GET /api/v1/health`.

4. Проверка целостности данных

4.1. Проверялись сериализация и десериализация клиентской сессии, транспортные контракты DTO и схемы серверных сущностей.

4.2. Проверялись X-Idempotency-Key, изоляция пользовательских ресурсов и отсутствие дублей при календарной синхронизации.

4.3. Проверялись корректность apply-плана, batch-sync и инвариантов fixed-событий при

ребалансировке.

5. Общий вывод по надежности

5.1. Проверенные сценарии подтверждают устойчивость критических частей программного комплекса к типовым ошибкам ввода, сетевым отказам и повторным операциям.

5.2. Ограничениями текущего этапа остаются отсутствие отдельного набора длительных нагрузочных испытаний и полного UI automation на устройстве.

5.3. Указанные ограничения не отменяют результата выполненного модульного, интеграционного и сквозного функционального тестирования, но должны учитываться при дальнейшем развитии тестовой базы проекта.

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Apple Developer Documentation. EventKit. // Электронный ресурс. URL: https://developer.apple.com/documentation/eventkit (дата обращения 29.04.26).
15. Apple Developer Documentation. SwiftUI. // Электронный ресурс. URL: https://developer.apple.com/documentation/swiftui (дата обращения 29.04.26).

16. Apple Developer Documentation. UserDefaults. // Электронный ресурс. URL: https://developer.apple.com/documentation/foundation/userdefaults (дата обращения 29.04.26).
17. Apple Developer Documentation. Keychain Services. // Электронный ресурс. URL: https://developer.apple.com/documentation/security/keychain_services (дата обращения 29.04.26).
18. Apple Developer Documentation. SFSafariViewController. // Электронный ресурс. URL: https://developer.apple.com/documentation/safariservices/sfsafariviewcontroller (дата обращения 29.04.26).
19. Swift.org. Swift Testing. // Электронный ресурс. URL: https://www.swift.org/documentation/testing/ (дата обращения 29.04.26).
20. Point-Free. The Composable Architecture. // Электронный ресурс. URL: https://github.com/pointfreeco/swift-composable-architecture (дата обращения 29.04.26).
21. pytest Documentation. [Электронный ресурс], URL: https://docs.pytest.org/ (дата обращения 20.04.2026).
22. httpx Documentation. [Электронный ресурс], URL: https://www.python-httpx.org/ (дата обращения 20.04.2026).
23. FastAPI Documentation. [Электронный ресурс], URL: https://fastapi.tiangolo.com/ (дата обращения 20.04.2026).
24. PostgreSQL Official Documentation. [Электронный ресурс], URL: https://www.postgresql.org/docs/ (дата обращения 20.04.2026).
25. Docker Documentation. [Электронный ресурс], URL: https://docs.docker.com/ (дата обращения 20.04.2026).

ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ

1. API (Application Programming Interface) — программный интерфейс взаимодействия клиента с серверной частью по HTTPS и JSON.
2. AppSettings — локальная структура пользовательских настроек приложения, включающая Default Prompt, выбранное окружение и сведения о недавно активном чате.
3. Default Prompt — пользовательский текстовый шаблон, автоматически добавляемый к каждому сообщению в чате и к каждому запросу ребалансировки.
4. EventKit — системный фреймворк Apple для доступа к календарям и событиям устройства.
5. Flow-тест — автоматизированный тест, проверяющий последовательность действий, эффектов и переходов состояния внутри пользовательского сценария.
6. JSON (JavaScript Object Notation) — формат представления структурированных данных, используемый в сетевых контрактах системы.
7. JWT (JSON Web Token) — токен аутентификации, используемый для авторизации клиента на сервере.
8. Keychain — защищённое системное хранилище Apple для токенов и иных чувствительных данных.
9. LLM (Large Language Model) — внешняя языковая модель, используемая серверной частью для интерпретации запросов и формирования предложений действий.
10. OpenAPI — формат описания REST API, используемый для документации серверной части.
11. Rebalance — функция Sentinel для получения и применения предложения по перераспределению событий на выбранном диапазоне дней.
12. Resource Battery — вычисляемая модель загруженности расписания пользователя на ближайший период или на отдельный день.
13. SFSafariViewController — системный компонент iOS для отображения веб-страниц внутри приложения.
14. Smoke-тест — быстрая проверка того, что экран или базовый сценарий может быть создан и отрисован без ошибки.
15. SwiftData — технология локального хранения данных, используемая для записей синхронизации календаря.
16. Swift Testing — фреймворк тестирования на Swift, применяемый в тестовом таргете iOS-клиента.
17. SwiftUI — декларативный фреймворк Apple для построения пользовательского интерфейса.

18. TCA (The Composable Architecture) — архитектурный подход с однонаправленным потоком данных, лежащий в основе клиентского приложения Sentinel.
19. TestStore — инструмент TCA для тестирования редьюсеров, их эффектов и переходов состояний.
20. UserDefaults — системное хранилище настроек пользователя, используемое для AppSettings и сервисных флагов.
21. UUID — универсальный уникальный идентификатор, используемый для доменных сущностей системы.
22. Vision — multimodal-сценарий LLM, при котором пользовательские изображения участвуют в обработке запроса.

ПРИЛОЖЕНИЕ 3. РЕЗУЛЬТАТЫ АВТОМАТИЧЕСКОГО ТЕСТИРОВАНИЯ BACKEND-КОМПОНЕНТА

Ниже приведён артефакт автоматизированного тестирования backend-компонента, использованный при подготовке исходного отдельного ПМИ для серверной части.

```
========================================================================== tests coverage ==========================================================================
_____________________________________________________________ coverage: platform darwin, python 3.10.15-final-0 _____________________________________________________________

Name                                                                    Stmts   Miss  Cover   Missing
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
app/__init__.py                                                             0      0   100%
app/api/__init__.py                                                         0      0   100%
app/api/dependencies.py                                                    19      3    84%   32-38
app/api/v1/__init__.py                                                      2      0   100%
app/api/v1/achievements.py                                                 25      9    64%   24-44, 60
app/api/v1/auth.py                                                        113     60    47%   40-58, 68-80, 91-108, 120-134, 146-155, 166-183
app/api/v1/chats.py                                                       114     63    45%   35, 67-72, 82-85, 98-117, 141-162, 169-171, 213-262
app/api/v1/events.py                                                       79     31    61%   39, 46, 59-67, 72-84, 103-113, 133-135, 147-150, 161-168
app/api/v1/rebalance.py                                                    38      7    82%   39-41, 51, 69-74
app/core/__init__.py                                                        0      0   100%
app/core/config.py                                                         51      2    96%   28, 31
app/core/limiter.py                                                        17      0   100%
app/core/schemas/__init__.py                                                3      0   100%
app/core/schemas/achievement.py                                            22      0   100%
app/core/schemas/chat.py                                                   74      0   100%
app/core/schemas/event.py                                                  63      0   100%
app/core/schemas/rebalance.py                                              32      0   100%
app/core/schemas/user.py                                                   41      0   100%
app/core/security.py                                                       28      0   100%
app/core/services/__init__.py                                               0      0   100%
app/core/services/achievement_service.py                                   47      0   100%
app/core/services/email_service.py                                         14      0   100%
app/core/services/llm_service.py                                          138     23    83%   125-127, 170-171, 175, 182-188, 196, 206-208, 265, 269-271, 287, 291, 293-294, 300, 302-303
app/core/services/rebalance_service.py                                    241     27    89%   76, 131, 148, 173-174, 281-282, 294, 296, 348, 361-372, 393, 405-406, 431, 440, 443, 449, 451
app/core/services/storage_service.py                                       12      0   100%
app/infrastructure/__init__.py                                              0      0   100%
app/infrastructure/database/__init__.py                                     0      0   100%
app/infrastructure/database/base.py                                        12      4    67%   31-35
app/infrastructure/database/models/__init__.py                              7      0   100%
app/infrastructure/database/models/achievement.py                          36      0   100%
app/infrastructure/database/models/auth_token.py                           15      0   100%
app/infrastructure/database/models/chat.py                                 26      0   100%
app/infrastructure/database/models/event.py                                22      0   100%
app/infrastructure/database/models/idempotency.py                          14      0   100%
app/infrastructure/database/models/user.py                                 23      0   100%
app/infrastructure/database/repositories/__init__.py                        6      0   100%
app/infrastructure/database/repositories/achievement_repository.py         28      0   100%
app/infrastructure/database/repositories/auth_token_repository.py          20      0   100%
app/infrastructure/database/repositories/chat_repository.py               114      0   100%
app/infrastructure/database/repositories/counter_repository.py             24      0   100%
app/infrastructure/database/repositories/event_repository.py               88      0   100%
app/infrastructure/database/repositories/idempotency_repository.py         22      0   100%
app/infrastructure/database/repositories/user_repository.py                45      0   100%
app/infrastructure/external/__init__.py                                     0      0   100%
app/infrastructure/external/openai_client.py                                4      0   100%
app/main.py                                                                43      1    98%   79
app/scripts/__init__.py                                                     0      0   100%
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
TOTAL                                                                    1722    230    87%
========================================================================== 382 passed, 1 skipped, 17 warnings in 54.63s ==========================================================================
% tim-maks@172 sentinel-backend %
```

Рис. 1. Отчёт о покрытии backend-кода тестами

Результат интерпретируется вместе с результатами unit- и integration-тестирования серверной части, описанными в разделе «Методы испытаний». При анализе покрытия необходимо учитывать техническое ограничение инструмента тестирования FastAPI-маршрутов, связанное с выполнением части кода в отдельном anyio-потоке.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					